Quantum Annealing with D-Wave Machines

Quantum Computing (M1)

Practical lab

Sloths[1] moon colonization



pic: © L.T.GraphicArtist

Real-life context. Fifty years had passed since the last animal went to the moon[2]. Lately, the sloths have shown their interest to colonize it[3]. Indeed, they stop being lazy today in order to be more lazy tomorrow. Their goal: to be weightless for the rest of their life.

Note: Since February 2025, free access to D-Wave machines has been blocked. To work around this issue while learning how to use this type of machine, you can use the Simulated Annealing metaheuristic instead of the Quantum Annealing from the D-Wave API. The API still needs to be installed, but creating a D-Wave account is no longer required. The code to be replaced is provided in this document. Part 2, written in this color, can be completed if the quantum machine becomes accessible again. For now, only parts 1 and 3 of this lab have to be done.

*To solve the problems presented in this document, you may also use **Pasqal's emulator**. Please note that you are expected to conduct your own research within the documentation to identify the functions required for **QUBO** (Quadratic Unconstrained Binary Optimization) inputs. **Note:** It is your responsibility to manually transform any Ising model formulation into a QUBO-compatible format.*

Official Documentation & Repositories:

- ***Pulser Framework**: GitHub Repo | Documentation*
- ***QUBO Solver**: GitHub Repo | Documentation*

[1] *fr: le paresseux (l'animal).*
[2] Apollo 17, taking place between 7 and 19 December 1972.
[3] For those who can be surprised by this news: so far, they were too lazy to show us they have the same intelligence.

Samuel Deleplanque. (prénom.nom@junia.com)

OVERVIEW

The infrastructure on the moon had been built. There are **2** different places for working and **3** separate dormitories for a total of **6** sloths. The remaining work[4] for this operation (i.e., **Part1**, **Part2 and Part3**) is about splitting the sloths group into subgroups and to calculate the best route for their vacation:

- **Part1**: During daytime, the group of 6 sloths must be split into 2 different groups for working: one for each of the 2 different locations. The separation will be made according to the first criterion **c1** defined below.
- **Part2**: During the night, the 6 sloths must be placed in one of 3 dormitories according to a second criterion **c2** and to a number of beds available in each dormitory.
- **Part3**: For their vacation, the sloths visit four of the most beautiful spots the moon has to offer. They need to find the route with the shortest distance to do so.

The two criteria (Part1 and Part2) can be defined and placed in their context:

- **c1: laziness risk.** Among the set of sloths, some pairs of them have a high risk of doing nothing if they are together to work. Thus, the separation of such pairs into two different groups (i.e., to work in different places) must be done as much as possible. For a given pair of sloths, the values related to the *laziness risk* can be defined as follow:
  - the more $c1_{ij}$ is high the more there is a risk of laziness if the sloths i and j work together.
- **c2: genetic heterogeneity.** The sloths will be also separated to create 3 groups: one per dormitory. These subgroups must be made in order to have genetic heterogeneity. For a given pair of sloths, the values related to the *heterogeneity* can be defined as follow:
  - the smaller the $c2_{ij}$ the more there is heterogeneïty if the sloths i and j sleep in the same dormitory.

The data related to the population and to the criteria **c1** & **c2** is given in the following section as well as the distances between the four spots to visit.

[4] actually: your work.

DATA

Because we are limited in time, we will work with a small population of **6** sloths. The data for Part1 (c1) is given in Figure 1. For Part2, Figures 2 and 3 give all the information: the values related to the criterion c2 and the number of beds per dormitory.

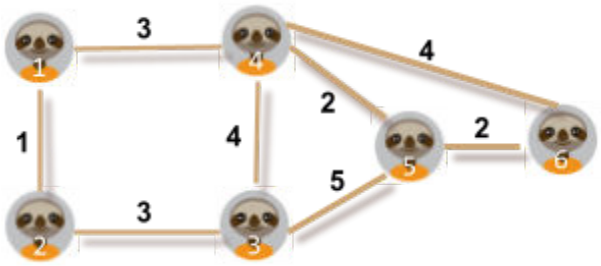


Figure 1. Laziness risk for Part1. Here, the valuation of each edge, if it exists, is the laziness risk: the bigger it is, the more the risk is high. For example, the sloths (3) and (5) have the highest risk of laziness.

i \ j	1	2	3	4	5	6
1	0	1	6	1	8	0
2		0	3	3	9	8
3			0	8	7	4
4				0	9	8
5					0	9
6						0

Figure 2. Genetic heterogeneity for Part2. Here: the lesser it is, the bigger the heterogeneity will be. This criterion is symmetric: $c2_{16} = c2_{61} = 0$ means the sloth 6 and the sloth 1 have the largest heterogeneity values while $c2_{25} = c2_{52} = 9$ means sloth 2 and 5 have the smallest genetic heterogeneity.

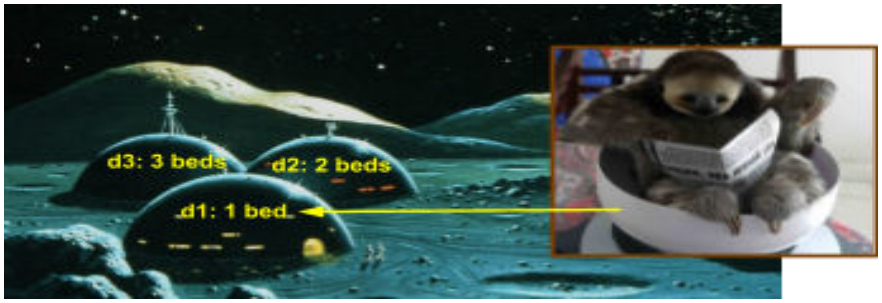


Figure 3. Capacities of the 3 dormitories d_1, d_2 and d_3

After so much hard work, the sloths are entitled to 50 weeks of vacation per year. Among the activities they can enjoy, a tour of the moon is offered. To practice calculating the best route, only four locations are considered for solving the traveling salesman problem (Part3).



Figure 4. Some Sloths Places to visit (pic: ChatGPT 4o)

All distances are provided in the following table:

Distances in kilometers	Sleepy Crater	Lethargic Quarter	Idle Area	Inert Mountain
Sleepy Crater	0	1100	890	230
Lethargic Quarter	1100	0	630	540
Idle Area	890	630	0	680
Inert Mountain	230	540	680	0

Figure 5. Distances between the four areas that must be visited by the Slothy Bus.

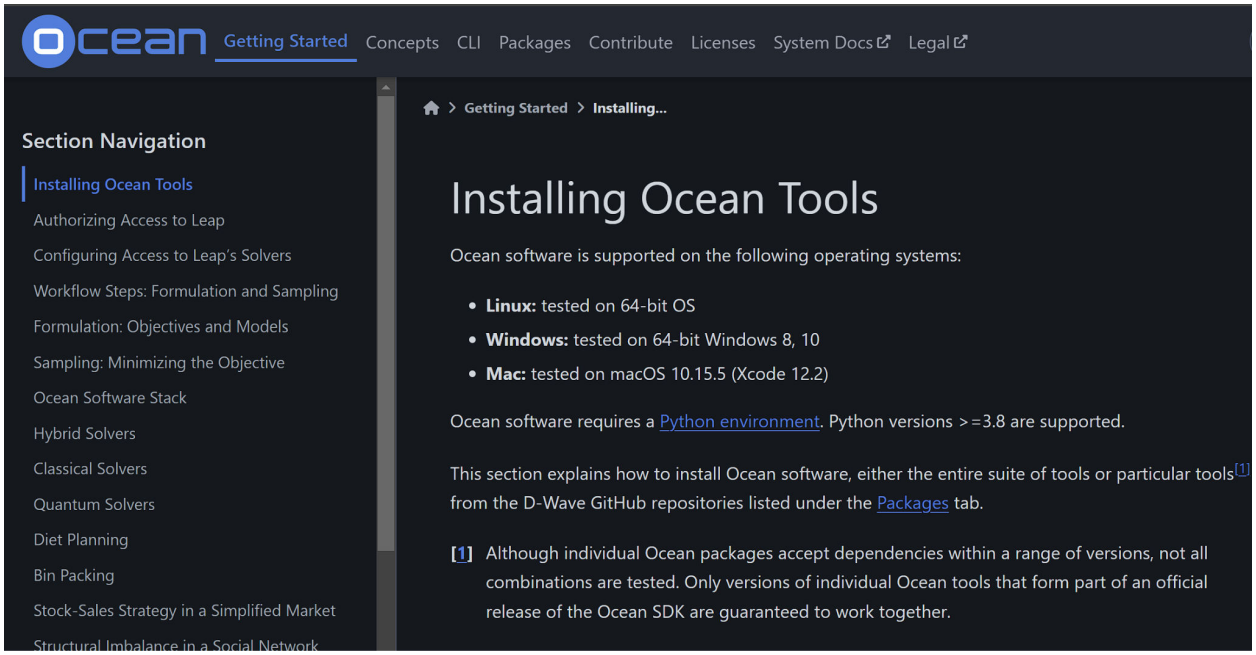
You can find helpful code and examples introduced in the course slides at:

- https://kdrive.infomaniak.com/app/share/223644/7bc30a76-157c-429d-9af9-55b115d250a9

Some help for installing Ocean (D-Wave) Tools

You can install the Ocean tools on Windows, Linux or MAC in order to use the DWave API. For the installation you can follow:

- https://docs.ocean.dwavesys.com/en/latest/overview/install.html



Few additional information:

- You might need to install virtualenv with '`pip install virtualenv`'
- On Windows the command for activating the virtual environment is a bit different (use Powershell):
    - `.\ocean\Scripts\activate`
    - If you have an error related to some rules which block the execution of some scripts:
        - still on Powershell (with super users rights) type:
            - set-executionpolicy unrestricted1
        - for installing the DWave sdk with the pip command as in[5] https://docs.ocean.dwavesys.com/en/latest/overview/install.html):

[5] You might need to upgrade your 'pip' command with python.exe -m pip install --upgrade pip

```
(ocean) PS C:\Users\sapla\Downloads> pip install dwave-ocean-sdk
Collecting dwave-ocean-sdk
  Downloading dwave_ocean_sdk-8.0.1-py3-none-any.whl.metadata (5.7 kB)
Collecting dimod==0.12.17 (from dwave-ocean-sdk)
  Downloading dimod-0.12.17-cp313-cp313-win_amd64.whl.metadata (4.2 kB)
Collecting dwave-cloud-client==0.13.1 (from dwave-ocean-sdk)
  Downloading dwave_cloud_client-0.13.1-py3-none-any.whl.metadata (5.5 k
B)
Collecting dwave-gate==0.3.2 (from dwave-ocean-sdk)
  Downloading dwave_gate-0.3.2.tar.gz (387 kB)
  Installing build dependencies ... \
```

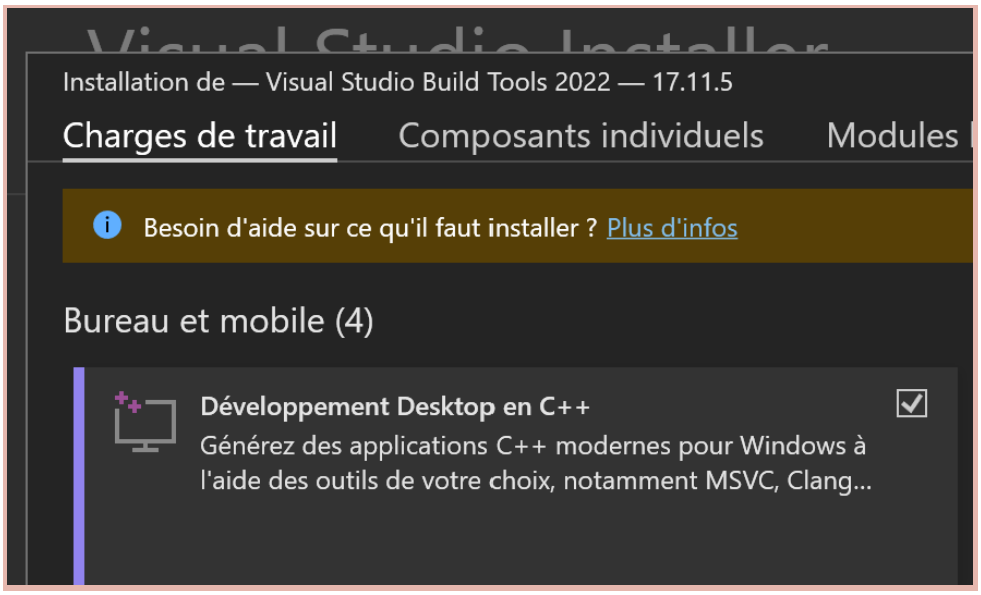
- (Windows) In case you have the following error, you will have to install https://visualstudio.microsoft.com/fr/visual-cpp-build-tools/ with the C++ tools checked:

```
  Preparing metadata (pyproject.toml) ... error
  error: subprocess-exited-with-error

  × Preparing metadata (pyproject.toml) did not run successfully.
  │ exit code: 1
  ╰─> [21 lines of output]
      + meson setup C:\Users\sapla\AppData\Local\Temp\pip-install-rk2g9x
v8\dwave-optimization_a6896cf46f004895b5809ebded2b0d18 C:\Users\sapla\Ap
pData\Local\Temp\pip-install-rk2g9xv8\dwave-optimization_a6896cf46f00489
5b5809ebded2b0d18\.mesonpy-epmp_4bc -Dbuildtype=release -Db_ndebug=if-re
lease -Db_vscrt=md --native-file=C:\Users\sapla\AppData\Local\Temp\pip-i
nstall-rk2g9xv8\dwave-optimization_a6896cf46f004895b5809ebded2b0d18\.mes
onpy-epmp_4bc\meson-python-native-file.ini
      The Meson build system
      Version: 1.6.0
      Source dir: C:\Users\sapla\AppData\Local\Temp\pip-install-rk2g9xv8
\dwave-optimization_a6896cf46f004895b5809ebded2b0d18
      Build dir: C:\Users\sapla\AppData\Local\Temp\pip-install-rk2g9xv8\
dwave-optimization_a6896cf46f004895b5809ebded2b0d18\.mesonpy-epmp_4bc
      Build type: native build
      Project name: dwave-optimization
      Project version: 0.3.0

      WARNING: Failed to activate VS environment: Could not find C:\Prog
ram Files (x86)\Microsoft Visual Studio\Installer\vswhere.exe
```

The following section is intended for D-Wave users but is not expected to be used while free access to D-Wave machines is unavailable.



- Authorize Access to Leap[6] with 'dwave auth login'
- Install the Install Contributor Ocean Tools as it is explained in the documentation. In particular, install: 'dwave install inspector'
- After installation you need to set up your environment. In short, after creating a DWave account[7] you will be able to link your local code with an online account. you will need to use a browser automatically launched with a command in powershell. Your local code will automatically be sent to a DWave machine online (generally in Canada).
    - Authorizing Leap access with 'dwave setup --auth'
- Finally, once you have activated your ocean environment, install all the required tools and authorized you access to Leap, you can run your python code with:
    - 'python yourPythonFile.py'

End of this section regarding unavailable machines.

[6] Leap is the quantum cloud service of D-Wave
[7] https://cloud.dwavesys.com/leap/signup/

Part1

i. Which optimization problem corresponds to part 1[8]?

ii. Formulate the Objective function from the graph given in Figure 1 by considering that the edges of the graph have no valuation (it is like to consider all integers on the edges equal to 1).

iii. Imagine one solution of the problem without necessarily finding the best one, and report it after calculating the Objective function you obtain for your solution. To report the solution, give the two sets of sloths.

Optimizing with an analog D-Wave machine (Part1).

iv. In the Python code provided below, you will find instructions corresponding to solving the problem in Part 1, but for a different case. Start by trying to run it after activating your Ocean environment. Use the course slides to help you understand the results displayed in the terminal.

[8] You have the slides on Teams…And not a lot of optimization problems to check…And you can also read the rest of this subject to be sure…

```python
import numpy as np
# Importer methodes pour definir entre autres un modèle ISING
import dimod
# On importe les librairies de DWAVE
from dwave.system.samplers import DWaveSampler
from dwave.system.composites import EmbeddingComposite

# Resolution du Max-Cut Problem sur une petite instance.

# On definit un group tres simple avec deux noeuds.
# J correspond aux expressions quadratiques
# (var1, var2):poids
J = {(1,2):1, (2,3):1, (3,4):1, (1,4):1, (4,5):1, (3,5):1, (4,6):1, (5,6):1}
# poids de chaque qubits pris individuellement
h = {}

# On rentre le modele dans la structure associee.
model = dimod.BinaryQuadraticModel(h, J, 0.0, dimod.SPIN)
# On affiche le modèle
print("Le modele resolu est le suivant :")
print(model)


##               RESOLUTION APPROCHEE              ##
##     Recuit quantique sur Machine D-WAVE         ##
sampler = EmbeddingComposite(DWaveSampler())
sampler_name = sampler.properties['child_properties']['chip_id']
response = sampler.sample(model, num_reads=8000)
print("The solution obtained by D-Wave's quantum annealer",sampler_name,"is")
print(response)
```

v. Using the portions of code provided, implement your python code for the problem of Part1 without valuation. You will find lots of help in the last slides of the Quantum Analog Machines course. Run your code on a D-Wave machine and report the results[9]. Also note the best solution obtained and find the detail of the calculation which gives the value of the Objective function (the energy column).

In case the quantum machine is not available, you can use the simulated annealing with the following code:

```python
sampler = SimulatedAnnealingSampler()

response = sampler.sample(model, num_reads=8000)
```

[9] With a screenshot of the resulting terminal.

vi. Carry out the same work but this time with the values of the arcs. This time: try to calculate the Max-Cut between the sets of sloths[10].

vii. Modify your code in order to solve with a more recent quantum computer by integrating the following instruction at the right place[11] :

```
sampler = EmbeddingComposite(DWaveSampler(solver='Advantage_system4.1'))
```

Report the result you obtain and check the differences or similarities with your previous result.

The following section is intended for D-Wave users but is not expected to be used while free access to D-Wave machines is unavailable.

---

viii. Show the initial graph of the problem and its transformation once it is mapped to the qubits graph. For that, you need to use the function `dwave.inspector`:

```
import dwave.inspector as inspector
from dwave.system import DWaveSampler, EmbeddingComposite

# blablacode

inspector.show(response)
```

Report what you obtain for two different machines with a different topology (i.e., a different qubit graph). Report screenshots of the results as it is done in the following figure related to a larger MaxCut instance.

[10] You will see some examples in the course: some slides represent such a cut with scissors and a dotted line. You just have to sum the values on the edges of the cut.

[11] In https://cloud.dwavesys.com/leap/, you have access to your Dashboard with some data related to the machines you can use. The Advantage 6.1 exists and other future machines like the one in beta test with 20-connectivity. Feel free to try other versions and to report it.

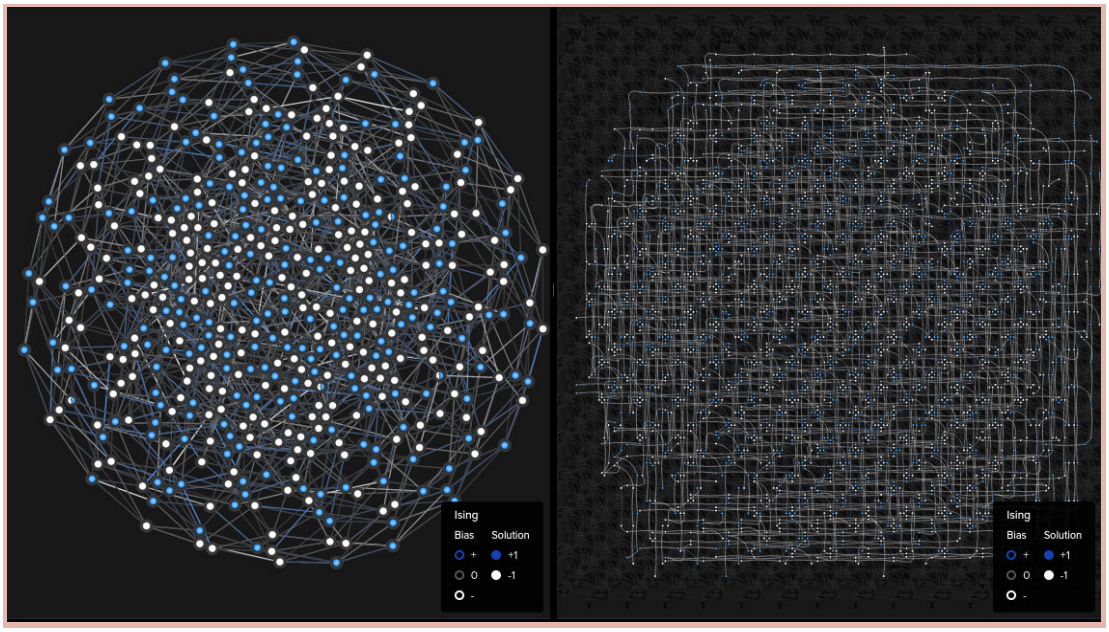


Figure 6. From left to right: a MaxCut graph and its mapping on the qubit graph.

End of this section regarding unavailable machines.

ix. In Quantum Annealing/Simulated Annealing, the *number of Shots* is called the *number of Reads*. This is not only related to the measure process, it is the number of samples. In your code, you can change the *number of Reads* (`num_reads=5000`). Try a few values (from a very small one to 5000) and analyze the result you obtain.

The following section is intended for D-Wave users but is not expected to be used while free access to D-Wave machines is unavailable.

Part2

Mathematical formulation of the problem. Maximizing the genetic heterogeneity of the 3 groups sleeping in different dormitories consists in creating 3 subgroups minimizing the total sum of all $c2_{ij}$ related to all the pairs (i,j) for slots in each group. To express that mathematically, we note d the index of the dormitory ($d = 1..3$), i and j the index of two sloths, and the constant 6 the total number of sloths. The x_{id} correspond to {0;1} binary variables[12] such that:

[12] So what we are looking for in this Part2.

- $x_{id} = 1$ if the sloth i is sleeping in the dormitory d, $x_{id} = 0$ otherwise.

The Objective function of the Part2, which corresponds of maximizing the genetic heterogeneity in the 3 dormitories, can be modeled by the minimization of the following function f[13].

$$Objective\text{: } Minimize\ (f) = \sum_{i=1}^{5} \sum_{j=i+1}^{6} \sum_{d=1}^{3} c2_{ij} x_{id} x_{jd}$$

The goal is to find which variables $x_{id} = 1$ in order to minimize f. However, it is not enough. Instead of the problem treated in Part1, this one requires constraints. Indeed, a feasible solution needs to satisfy two series of them, where we note $Beds_d$ the number of available beds in the dormitory d:

1. Bed number limit in each dormitory. It can be modeled as follow with 3 constraints:
   - $\sum_{i=1}^{6} x_{id} \leq Beds_d, \qquad \forall\, d = 1..3$
2. Each sloth must be linked to exactly one dormitory. It can be modeled as follow with 6 constraints:
   - $\sum_{d=1}^{3} x_{id} = 1, \qquad \forall\, i = 1..6$

x. Take the time to write the Objective function and all the constraints you obtain for the case related to the data given above (Figure 2 and 3). Since the mathematical program is long, you can write it by hand, and take a picture for your report.

xi. Without necessarily finding the best solution, imagine one solution of the problem according to the model (Objective function and constraints) and the data given above. Report the variables values of all the x_{id}, the resulting Objective function, and the impact of the solution in all the constraints[14].

Optimizing with an analog D-Wave machine (Part2).

Lately[15], it is possible to declare a quadratic model with constraints directly in the python code[16]. For the problem treated in this Part2 the way to declare our problem is quite different from the problem of Part1. Here we will not care about the Hamiltonien (J and h in miniMaxCut.py) since it is possible to directly formulate the Objective function and the constraints in the code.

xii. Write the mathematical model (1/2).

Create a new file from scratch directly in the online Leap IDE. Import the following packages[17] and the declaration of our model object[18]:

```
from dimod import ConstrainedQuadraticModel, CQM,  SampleSet

from dimod import Binary, quicksum

from dwave.system import LeapHybridCQMSampler

import numpy as np
```

Note that the type of the variables is furnished by the package `Binary` and the different sums of the Objective function and the constraints will be written with the `quicksum` function.

---

[13] If you have trouble understanding that j starts at i+1, try to link that with the fact the table given in Figure 2 is symmetric. Why would we do the double calculation for nothing?

[14] e.g., for the first dormitory constraint you could report: 0 + 0 + 1 + 0 + 0 + 0 <= 1 or, e.g., for the first constraints of the second series: 0 + 1 + 0 = 1.

[15] end of 2021.

[16] Before that, all the constraints must have been relaxed. In short: in a relaxation process, all the constraints are integrated in the Objective function in a way that, if they are not satisfied, the Objective function will tend to take a value in the contrary direction of the optimization (penalties). The difficulty is to weigh each constraint in the Objective function. D-Wave proposes to do it, but it does not mean they succeed all the time.

[17] You may notice that you can use numpy!

[18] Advice: rewrite the code or at least redo the tabulation

Integrate the data and declare the model object `cqm`.

Use the same data structures as the ones given below. You just have to adapt them to our case (e.g., you have 6 sloths instead of 4).

```
totalNumberOfSloths = 4

numberOfDormitories = 2

c2 = [[0, 3, 4, 10], [3, 0, 2, 2], [4, 2, 0, 3], [10, 2, 3, 0]]

beds = [2, 2]

cqm = ConstrainedQuadraticModel()
```

Declare the variables.

We have seen that x_{id} is the binary variable for the sloth i and the dormitory d. To declare the 6*3 matrix for all the variables, you can use the following code:

```
x = {
    (i, d): Binary('x{}_{}'.format(i, d))
            for i in range(totalNumberOfSloths)
            for d in range(numberOfDormitories)}
```

Declare and furnish the Objective function.

The following code is a "draft" to declare the Objective function:

```
objective = quicksum(c2[i][j] * x[(i,d)] * x[(j,d)]
            for i in range(totalNumberOfSloths)
            for j in range(totalNumberOfSloths)
            for d in range(numberOfDormitories)  )
cqm.set_objective(objective)
```

xiii. Modify the Objective function according to the mathematical model.

Keep the same last instruction to set the Objective function to the problem (i.e., to `cqm`).

xiv. Write the mathematical model (2/2).

Declare the bed number limit constraints.

To add a constraint to our qcm object, there is the `add_constraint` function:

```
for d in range(numberOfDormitories):
    cqm.add_constraint(quicksum(x[(i,d)]
    for i in range(totalNumberOfSloths)) <= beds[d])
```

Declare the dormitory affectation constraints.

```
for i in range(totalNumberOfSloths):
```

```
        cqm.add_constraint(quicksum(x[(i,d)]

        for d in range(numberOfDormitories)) == 1)
```

Launch the calculation.

In the following code, the call of the constructor `LeapHybridCQMSampler` gives the information that the computer used is hybrid. Instead of the machine used for Part1 which was a full quantum machine, here, the work is made between a quantum and a classical machine. Click on the documentation link for more information on `sample_cqm`.

```
cqm_sampler = LeapHybridCQMSampler()

sampleset = cqm_sampler.sample_cqm(cqm)
```

xv. Display and analyze the results

To display your results, use the following instruction:

```
print(sampleset)
```

To go further: try to find the right functions in the documentation in order to display the full model (Objective & Constraints), and all the solution, especially here:

- https://docs.ocean.dwavesys.com/en/stable/docs_dimod/reference/sampleset.html#dimod.SampleSet. For instance if you are having difficulty showing all the results, `print(sampleset.lowest)` can help you (but not only).

You obtain the results for a certain number of solutions. You have to report:

- one screenshot of the result (even if it takes one page or more in your report),
- analyze the result and give one of the best and feasible[19] solutions obtained,
- draw simply the 3 subgroups of sloths in their dormitory,
- you might rapidly understand that some solutions cannot be taken into account. Take one and explain why.

(Bonus). If that was not enough work.

If you have more time, you might want to use the potential of one of the D-Wave quantum machines. You just need to generate a case on a wider scale (e.g., 50 or 100 sloths) or take instances from the literature[20].

End of this section regarding unavailable machines.

---

[19] i.e., respecting all the constraints.

[20] The figure 6 had been obtained from the resolution of the MaxCut problem instances from the set2 that you can find through the following URL: https://grafo.etsii.urjc.es/optsicom/maxcut.html#instances

Part3



xvi. The lazy sloth's tour problem. Even though this part is likely the most complex, it won't take the longest to define. With the course material and this document, you have everything you need to solve the traveling salesman problem for the sloths' tour of the four spots on the moon. You need to write the code for a fully quantum machine (without hybrid, using a BQM instead of a CQM, as covered in the course), run it, and include your program along with screenshots of the results (including the inspector graph representation) in your report.

xvi. Asimovsloth Foundation. A few centuries have passed… Under the reign of Cleon the Zzz'er, a few rebellious sloths, including Hari Slothdin and Gaal Sloarnick, have sought to challenge the empire. They plan to establish several new colonies on the moon. To do so, they form a team of renowned explorers (Christopher Colossloth, Vasco da Slothma, Maggellanthropus, and Neil Slowstrong). They aim to establish around ten colonies, but the distances between them are so vast that they must determine the best route to connect them. Their computer, quantum yet sluggish, likely won't allow them to reach a dozen sites in their search for a good (if not optimal) solution to the traveling salesman problem. Using the instance generator (`tsp_file_generation.py`), it's up to you to establish the maximum number of cities that can be processed with their centuries-old machine: the Advantage machine. It's also up to you to explain your method in your report and include all elements needed for a proper evaluation of your work.



pic: ChatGPT 4o

CONCLUSION

- Of course we could not reach a problem size that makes the quantum annealing computer competitive: too much data would have had to be manipulated and too much time would have had to be spent doing it. This type of machines has today some limits: the number of qubits is not bad, but the connectivity between them is low[21]. For example for 20 sloths, the connectivity of 15 of the Advantage D-Wave machine will not be enough (some costly operations will use plenty of qubits to simulate a 19 connectivity).

- You might have realized that the two problems considered in this work have quadratic formulations (here: the products of binaries). It is for a good reason: the quantum machines based on quantum annealing can handle "easily" such expressions while the process implemented with this type of formulation for running on classical computers is way more complex. Pasqal machines are another type of analog machines working on transverse-field Ising models (Ising models and QUBOs -Quadratic Unconstrained Binary Optimization- are isomorphic). Moreover, there are optimization algorithms that can be implemented on universal gated-based quantum computers (e.g., the IBM Osprey). The most promising today is the QAOA algorithm[22] (there is a Qiskit implementation[23]) and it also takes QUBOs as input.







[21] But much more important than universal quantum gates machines (IBM, Google etc.).

[22] Farhi, E., Goldstone, J., & Gutmann, S. (2014). A quantum approximate optimization algorithm. arXiv preprint arXiv:1411.4028.

[23] https://qiskit.org/textbook/ch-applications/qaoa.html

- You worked on 3 optimization problems: a max-cut problem, a clustering problem and the traveling salesman problem. The sloths are not the only reasons to know how to solve them. For example, for the clustering problem (about the beds), it is the same problem when you want to assign packages to a fleet of delivery vehicles (the number of beds is the capacity of the vehicles). Moreover, you already know how to solve the Traveling Salesman Problem, so you have good tools to begin the management of a fleet of delivery vehicles.

ABOUT the REPORT & the CODE to return:

- the number of pages is (here) not important, just try to answer each question,
- it can be written in french or english; screenshots are welcomed,
- the type of the file must be .pdf,
- 4 files not zipped: the report and 3 code files (resp. for Part1, Part2 and Part3). Rename the '.py' files into '.txt'.
- send the files by email,
- the deadline will be given soon, or you just got it!



pic: © https://memegenerator.net/instance/68420692/sarcastic-sloth-i-just-wanted-to-say-goodbye